

In-House Development of LiDAR-Based AVC Systems

Detailed Technical Documentation & Implementation Reference

Akshat Gupta
IIT Mandi x EFKON India Pvt Ltd

January 14, 2026

Contents

Introduction	3
1 Phase 1: Sensor Integration & Data Acquisition	4
1.1 Sensor Selection & Comparative Analysis	4
1.1.1 YDLidar TG30 (360 Scanning)	4
1.1.2 BEA LZR-U921 (Fixed Curtain Profiling)	4
1.2 BEA LZR-U921 Interface	5
1.2.1 Connection & Configuration Parameters	5
1.2.2 Binary Protocol Parsing	5
1.2.3 Coordinate Transformation	5
2 Phase 2: Preprocessing & Background Subtraction	7
2.1 Robust Zero Plane Calibration	7
2.1.1 Calibration Parameters	7
2.1.2 Statistical Filtering (MAD)	7
2.2 Real-Time Filtering State Machine	7
2.2.1 Detection Logic	8
2.3 Noise Removal Pipeline	8
3 Phase 3: Representation Reconstruction	9
3.1 3D Reconstruction	9
3.2 Image Generation Pipeline	9
3.2.1 3D Rotation Correction	9
3.2.2 Dynamic Framing	9
3.2.3 BEV (Bird's Eye View) Projection Logic	10
4 Phase 4: Deployment Field Validation	11
4.1 Hardware Configuration	11
4.1.1 Software Environment	11
4.1.2 Edge-Specific Optimizations	11
4.2 On-Site Deployment	11
4.2.1 Site Details	11
4.2.2 Robustness Testing	12
4.2.3 Field Observations: Speed vs. Quality	12
4.2.4 Hardware Limitations: Image Breakdown	12
4.2.5 Challenges with Dynamic Speed Estimation	12
5 Phase 5: Comprehensive Code Walkthrough	14

5.1	Block 1: Configuration & Constants	14
5.2	Block 2: Statistical Helper (Robust Median)	15
5.3	Block 3: The Background Processor (Consumer Thread)	15
5.4	Block 4: Main System Class (TollPlazaSystem)	16
5.5	Block 5: Main Loop (State Machine)	17

Introduction

This document details the complete technical implementation of the LiDAR-based Automatic Vehicle Classification (AVC) system. The core logic documented herein corresponds to the `master_control_rpi.py` executable, which is the final validated code running on the Raspberry Pi edge unit at the toll plaza.

The system uses a BEA LZR-U921 sensor to capture high-resolution side profiles of passing vehicles, reconstructs them in 3D, and generates dynamic 2D images for classification.

Phase 1: Sensor Integration & Data Acquisition

Objective: Sensor selection, low-level driver implementation, and raw data parsing.

1.1 Sensor Selection & Comparative Analysis

The project evaluated two distinct types of LiDAR sensors. The choice of sensor fundamentally dictates the reconstruction methodology suitable for the AVC system.

1.1.1 YDLidar TG30 (360 Scanning)

- **Mechanism:** Mechanical rotation (360-degree spin).
- **Data Output:** Point cloud of the entire surrounding environment (Panoramic).
- **Limitation for AVC:** Since the sensor head rotates physically, it captures the scene in a spiral pattern. For a vehicle moving at high speed (e.g., 40 km/h), the vehicle moves significantly *during* a single rotation. This introduces severe "motion blur" and spiral distortion, making it difficult to reconstruct a solid 3D shape without complex motion compensation algorithms.

1.1.2 BEA LZR-U921 (Fixed Curtain Profiling)

- **Mechanism:** Solid-state / Fixed laser curtain (4 planes).
- **Data Output:** A single, high-frequency 2D slice of the object passing through the curtain.
- **Methodology Fit:** This sensor is ideal for the **Time-Stacked Reconstruction** method. Since the sensor is stationary, every "slice" corresponds to a specific timestamp. By stacking these slices along the time axis (Z-axis) as the vehicle moves through the curtain, we obtain a distortion-free 3D profile, similar to a CT scan.

Conclusion: The LZR-U921 was selected for the final deployment as its high scan rate (921,600 baud) and fixed geometry allow for precise slicing of fast-moving vehicles, which is the core requirement for the side-profiling solution.

1.2 BEA LZR-U921 Interface

Source: `master_control_rpi.py`

The sensor is interfaced via USB (mapped to `/dev/ttyUSB0`) using a robust threaded serial reader to handle high-bandwidth data streams without blocking the main processing loop.

1.2.1 Connection & Configuration Parameters

To ensure adaptability across different hardware setups and toll plaza geometries, the following parameters are configurable:

- **Port:** `/dev/ttyUSB0`
Note: This value is device-dependent. On the Raspberry Pi, it may enumerate as `/dev/ttyACM0` or `/dev/ttyUSB1` depending on other connected peripherals. The correct port must be verified using `dmesg` or `ls /dev/tty*` before execution.
- **Baud Rate:** 921,600 bps
Required to support the high data throughput of the LZR-U921.
- **Timeout:** 0.1 seconds
Configured for non-blocking reads to prevent the serial thread from hanging.
- **Lane Width (Max Range):** `MAX_RANGE_M = 2.25m`
This parameter acts as a software filter to define the active lane width. Points detected beyond this distance (e.g., adjacent traffic, toll booths) are automatically discarded. This value can be modified in the code to match the specific physical width of the toll lane.

1.2.2 Binary Protocol Parsing

The `get_raw_frame()` function continuously polls the serial buffer for a specific synchronization header.

- **Sync Header:** `0xFC 0xFD 0xFE 0xFF`
- **Packet Structure:**
 1. Header (4 bytes)
 2. Payload Size (2 bytes, Little-Endian)
 3. Message Body (Size bytes)
 4. Checksum (2 bytes)
- **Message ID Filter:** The system strictly processes Command ID 50011 (Measurement Data).

1.2.3 Coordinate Transformation

Raw distance data (millimeters) is converted to Cartesian meters using the specific angular geometry of the LZR-U921 sensor:

- **Start Angle:** -48.0°
- **Angular Resolution:** 0.3516°

For index i in the raw data array:

$$\theta_i = \text{rad}(-48.0 + i \times 0.3516)$$

$$x_i = (d_i/1000.0) \times \cos(\theta_i)$$

$$y_i = (d_i/1000.0) \times \sin(\theta_i)$$

Phase 2: Preprocessing & Background Subtraction

Objective: Real-time segmentation of moving vehicles from static infrastructure.

2.1 Robust Zero Plane Calibration

Source: `master_control_rpi.py`

Before operation, the system builds a "Background Matrix" representing the static environment (road, dividers, walls).

2.1.1 Calibration Parameters

- **Duration:** 4000 Frames.
- **Grid Cell Size:** 0.05 meters (5 cm resolution).

2.1.2 Statistical Filtering (MAD)

To prevent transient noise during calibration, the system uses the **Median Absolute Deviation (MAD)** algorithm instead of a simple mean.

1. Collect history of distances for each beam index over 4000 frames.
2. Compute Median M .
3. Compute MAD: $\text{median}(|x_i - M|)$.
4. **Filter:** Retain only points where $|x_i - M| \leq 3 \times \text{MAD}$.

2.2 Real-Time Filtering State Machine

The main loop processes every incoming frame through a state machine to detect vehicle presence.

2.2.1 Detection Logic

- **Background Check:** A point (x, y) is considered "foreground" if its grid cell $(x//0.05, y//0.05)$ is NOT in the calibrated set.
- **Trigger Threshold:** A frame is considered "active" if it contains > 50 foreground points.
- **Idle Timeout:** Recording stops if no active frames are detected for 0.8 seconds.

2.3 Noise Removal Pipeline

Once a vehicle is captured, the accumulated point cloud is cleaned using Open3D libraries optimized for the Raspberry Pi.

1. Statistical Outlier Removal (SOR):

- Neighbors: 25
- Std Ratio: 0.8

2. Radius Outlier Removal (ROR):

- Min Points: 12
- Radius: 0.06m

Phase 3: Representation Reconstruction

Objective: Generating high-quality 2D images for Machine Learning.

3.1 3D Reconstruction

The 2D slices are stacked along the Z-axis (Time) to form a 3D shape.

- **Estimated Velocity:** 5 km/h $\approx$ 1.38 m/s.
- **Z-Calculation:** $Z = (t_{current} - t_{start}) \times 1.38$.

3.2 Image Generation Pipeline

Source: background_processor in master_control_rpi.py

3.2.1 3D Rotation Correction

To correct for the physical mounting tilt of the sensor, a rotation matrix is applied to the point cloud before image generation.

- **Pitch (X-Axis):** 25.0°
- **Roll (Y-Axis):** 0.0°
- **Yaw (Z-Axis):** 0.0°

3.2.2 Dynamic Framing

Instead of a fixed canvas, the system dynamically calculates the bounding box for every vehicle to maximize resolution.

1. **Compute Bounds:** Find min/max (x, y) of the rotated cloud.
2. **Padding:** Add 0.2m buffer on all sides (`FRAME_PADDING_M`).
3. **Minimum Size:** Enforce a minimum dimension of 1.0m to prevent errors on small objects.

3.2.3 BEV (Bird’s Eye View) Projection Logic

To convert the unstructured 3D point cloud into a structured 2D image, the system employs an **Orthographic Grid Projection**, commonly referred to in LiDAR pipelines as BEV projection. Although applied here to generate a side profile, the underlying algorithm is identical to top-down map generation.

1. Grid Discretization: The continuous metric coordinates (x, y) of the point cloud are mapped to discrete pixel indices (c, r) based on the defined resolution.

$$c = \lfloor \frac{x - x_{min}}{GRID_RES} \rfloor, \quad r = \lfloor \frac{y - y_{min}}{GRID_RES} \rfloor$$

- **Resolution:** $GRID_RES = 0.005\text{m}$ (5mm/pixel).

- **Canvas Initialization:** A zero-matrix (black) is initialized with dimensions $(Width_{px}, Height_{px})$.

2. Intensity Encoding (Depth Map): The 3rd dimension (Depth/Distance from sensor) is encoded into the pixel’s grayscale intensity. This preserves 3D geometric information in the 2D image.

$$I_{pixel} = 1.0 - \min\left(\frac{d_{point}}{50.0}, 1.0\right)$$

where d_{point} is the Euclidean distance of the point from the sensor origin. This creates an image where closer parts of the vehicle appear brighter (white) and distant parts appear darker.

3. Morphological Post-Processing: Due to the vehicle’s motion and the LiDAR’s scanning frequency, raw projections often contain vertical gaps between scan lines. A Gaussian Blur kernel is applied to fill these gaps:

```
1 # Flip Y-axis so ground is at the bottom
2 bev_img = np.flipud(bev_img)
3 # Apply Gaussian Blur to close gaps
4 bev_img = cv2.GaussianBlur(bev_img, (3, 3), 0)
```

Phase 4: Deployment Field Validation

4.1 Hardware Configuration

The final code is deployed on a Raspberry Pi 4 Model B acting as the edge compute unit.

4.1.1 Software Environment

To ensure stability and prevent dependency conflicts on the ARM64 architecture, the following specific library versions are used:

- **Operating System:** Raspberry Pi OS (64-bit)
- **Python Version:** 3.11.10
- **OpenCV:** 4.10 (Headless build)
- **NumPy:** 1.26.4
- **Open3D:** open3d-cpu (CPU-only build for ARM)

4.1.2 Edge-Specific Optimizations

- **Memory Safety:** Point cloud arrays are explicitly cast to `float64` to prevent segmentation faults in Open3D C++ bindings.
- **Headless Operation:** Visualization calls are wrapped in checks for `DISPLAY` environment variables to allow background execution via SSH.

4.2 On-Site Deployment

Source: Project Presentation & Field Reports

4.2.1 Site Details

The system was physically installed and tested at the **Jewar Toll Plaza**.

- **Mounting:** The LZR-U921 was mounted on the toll lane gantry to capture the side profiles of vehicles passing through the lane.

- **Integration:** The Raspberry Pi was powered via a standard 5V adapter, with the LiDAR sensor connected via the RS-485 to USB interface.

4.2.2 Robustness Testing

To validate system stability under continuous load, a long-duration endurance test was conducted:

- **Test Window:** 7:00 PM to 2:00 AM (7 Hours Continuous).
- **Outcome:** The system successfully processed vehicles without crashing or memory leaks, validating the threaded architecture and memory management on the Raspberry Pi.
- **Environmental Factors:** The system operated effectively during night hours, confirming that the LiDAR (active sensor) is unaffected by ambient lighting conditions.

4.2.3 Field Observations: Speed vs. Quality

Field testing revealed a direct correlation between vehicle speed and the quality of the reconstructed 3D images, largely governed by the LZR-U921's scan frequency.

1. **Optimal Performance (20 km/h):** The deployed solution (`master_control_rpi.py`) produced consistent, high-fidelity images for vehicles moving at speeds around **20 km/h**. At this speed, the fixed-velocity reconstruction model aligned scan lines perfectly without aspect ratio distortion.
2. **High Speed Compression (>40 km/h):** At significantly higher speeds, the reconstructed images appeared compressed along the horizontal axis (aliasing) due to the sensor's finite scan rate.

4.2.4 Hardware Limitations: Image Breakdown

During testing, occasional "image breakdown" or segmentation artifacts were observed where the vehicle body appeared fragmented or disjointed in the final image. This phenomenon was traced to the limitations of the edge hardware:

- **Buffer Overflow:** The high baud rate (921,600 bps) generates a byte stream that occasionally fills the Raspberry Pi's UART serial buffer faster than the CPU can process it.
- **Processing Latency:** When the CPU is under load (e.g., performing matrix rotations or writing images to disk), it may briefly pause reading from the buffer. This causes a desynchronization in the timestamping of scan lines, leading to visible gaps or "breaks" in the reconstructed vehicle body.

4.2.5 Challenges with Dynamic Speed Estimation

An attempt was made to implement dynamic speed estimation using frame-to-frame centroid tracking (DBSCAN + Kalman Filter). However, this approach failed due to the specific "light curtain" geometry of the LZR-U921 sensor.

- **The Slicing Problem:** The sensor is stationary and captures 2D slices (e.g., bumper, then hood, then roof) rather than tracking a moving 3D volume. Calculating the centroid difference between two frames measures the change in vehicle shape (cross-section), not the vehicle's longitudinal speed.
- **Lack of Longitudinal Data:** The sensor measures distance (Y) and height (X) but has no depth perception along the road (Z). Movement along the travel axis is invisible between single frames.
- **Result:** The centroid tracking code produced erratic speed values caused by shape variations ("jitter") rather than actual motion. Consequently, the fixed-velocity reconstruction model proved more robust for generating recognizable vehicle profiles.

Phase 5: Comprehensive Code Walk-through

Objective: Detailed line-by-line analysis of the final deployed script.

This section provides a complete breakdown of the `master_control_rpi.py` script. The code is divided into four main logical blocks: Configuration, Helper Functions, Background Processing, and the Main System Class.

5.1 Block 1: Configuration & Constants

This block defines the static parameters that control the system's behavior. These are placed at the top of the file for easy field tuning.

```
1 import serial
2 import struct
3 import numpy as np
4 import time
5 import cv2
6 import open3d as o3d
7 import threading
8 import queue
9 import os
10 from datetime import datetime
11 import statistics
12
13 # =====
14 # CONFIGURATION
15 # =====
16 SERIAL_PORT = '/dev/ttyUSB0' # Physical USB Port
17 BAUD_RATE = 921600           # High speed for LZR-U921
18
19 # Sensor Properties (LZR-U921 Specifics)
20 START_ANGLE = -48.0
21 ANGULAR_RES = 0.3516
22
23 # Calibration
24 CALIBRATION_FRAMES = 4000      # 4000 frames used for Zero Plane
25 GRID_CELL_SIZE = 0.05         # 5cm grid cells for background lookup
26
27 # Filtering
28 MIN_RANGE_M = 0.10
```

```
29 MAX_RANGE_M = 2.25 # Max lane width (configurable)
```

Walkthrough:

- **Imports:** `serial` is used for the LZR-U921 connection. `open3d` handles 3D point cloud operations. `threading` and `queue` enable the Producer-Consumer architecture, allowing the serial reader to run uninterrupted while heavy image processing happens in the background.
- **BAUD_RATE (921600):** Essential for the LZR-U921. Standard 9600/115200 rates are insufficient for the data volume.
- **MAX_RANGE_M (2.25):** This is the "Software Lane Boundary". Any point detected further than 2.25m is ignored, effectively filtering out adjacent lane traffic or toll booths.

5.2 Block 2: Statistical Helper (Robust Median)

```
1 def robust_median(distances):
2     if len(distances) < 5: return None
3     median = statistics.median(distances)
4     deviations = [abs(d - median) for d in distances]
5     mad = statistics.median(deviations)
6     if mad == 0: return median
7     # Filter points > 3 * MAD
8     filtered = [d for d in distances if abs(d - median) <= 3 * mad]
9     return statistics.median(filtered) if filtered else None
```

Walkthrough:

- This function is critical for the **Zero Plane Calibration**.
- It calculates the **Median Absolute Deviation (MAD)** instead of Standard Deviation.
- **Reasoning:** Standard deviation is heavily skewed by outliers (e.g., a person walking through the lane during calibration). MAD is robust against such anomalies, ensuring the "Zero Plane" represents the true static ground.

5.3 Block 3: The Background Processor (Consumer Thread)

This function runs in a separate thread. It takes raw vehicle slices from the queue and converts them into the final image.

```
1 def background_processor():
2     while True:
3         extracted_stack, ts = processing_queue.get()
4         # ... folder creation code ...
5
6         # 1. Cleaning (Statistical Outlier Removal)
7         pcd = o3d.geometry.PointCloud()
8         # Explicit float64 cast for ARM architecture stability
```

```

9      pcd.points = o3d.utility.Vector3dVector(np.array(
extracted_stack, dtype=np.float64))
10      pcd, _ = pcd.remove_statistical_outlier(nb_neighbors=25,
std_ratio=0.8)
11
12      # 2. Rotation Correction (3D Mounting Compensation)
13      rad_x = np.radians(ROTATION_X_DEG) # Pitch (25 deg)
14      # ... Rotation Matrix construction (Rx, Ry, Rz) ...
15      R_total = Rz @ Ry @ Rx
16      pts_side = (R_total @ pts_side.T).T
17
18      # 3. Dynamic Framing
19      min_x, min_y = np.min(pts_side[:, :2], axis=0)
20      max_x, max_y = np.max(pts_side[:, :2], axis=0)
21      # Apply Padding (20cm)
22      min_x -= FRAME_PADDING_M; max_x += FRAME_PADDING_M
23
24      # 4. Image Generation (Grid Projection)
25      xi = ((pts_side[:, 0] - min_x) / GRID_RES).astype(np.int32)
26      yi = ((pts_side[:, 1] - min_y) / GRID_RES).astype(np.int32)
27
28      # Intensity Mapping (Depth -> Brightness)
29      dist = np.sqrt(v_points[:, 0] ** 2 + v_points[:, 1] ** 2)
30      intensity = 1.0 - np.minimum(dist / MAX_DIST_INTENSITY, 1.0)
31
32      # Save Final Image
33      bev_img = np.flipud(bev_img) # Correct orientation
34      cv2.imwrite(img_path, bev_img)
35      processing_queue.task_done()

```

Walkthrough:

- **Thread Safety:** It uses `processing_queue.get()` to wait for data. This decouples processing from the serial reader to prevent buffer overflows.
- **ARM Optimization:** `dtype=np.float64` is explicitly used. Open3D on Raspberry Pi (ARM64) can segmentation fault if passed default float32 arrays from NumPy.
- **Rotation:** The `R_total` matrix aligns the vehicle. Without this, the vehicle would look tilted corresponding to the sensor's mounting angle (25°).
- **Dynamic Framing:** Instead of a fixed 1000×1000 image, it calculates `min_x` and `max_x` to create a tight crop around the vehicle, improving resolution efficiency.

5.4 Block 4: Main System Class (TollPlazaSystem)

```

1 class TollPlazaSystem:
2     def __init__(self):
3         # Open Serial Port
4         self.ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=0.1)
5         self.background_matrix = set() # 0(1) Lookup for background
6
7     def get_raw_frame(self):
8         # 1. Sync Header Detection

```

```

9         if self.ser.read(1) == b'\xfc' and self.ser.read(3) == b'\xfd\
           xfe\xff':
10             # 2. Read Payload Size (Little Endian <H)
11             size = struct.unpack('<H', self.ser.read(2))[0]
12             # 3. Read Body & Verify ID 50011
13             body = self.ser.read(size)
14             if struct.unpack('<H', body[:2])[0] == 50011:
15                 # 4. Parse Distances
16                 d_mm = struct.unpack('<H', data[i*2 : i*2+2])[0]
17                 # Convert Polar -> Cartesian
18                 pts.append([dist * np.cos(ang), dist * np.sin(ang)])

```

Walkthrough:

- **get_raw_frame:** This is the lowest-level driver. It scans the byte stream for 0xFC 0xFD 0xFE 0xFF. This header synchronization is vital; without it, the system would read garbage values in the middle of a packet.
- **Message ID 50011:** The code specifically checks for this ID, ensuring we are interpreting "Measurement Data" and not debug logs or heartbeats from the sensor.

5.5 Block 5: Main Loop (State Machine)

```

1     def run_forever(self):
2         # Start Background Thread
3         t = threading.Thread(target=background_processor, daemon=True)
4         t.start()
5
6         self.calibrate() # Build Zero Plane
7
8         while True:
9             raw_xy = self.get_raw_frame()
10            # Z-axis calculation based on Time
11            z = (time.time() - start_time) * VEHICLE_SPEED_MPS
12
13            # Background Subtraction
14            for p in raw_xy:
15                if (grid_x, grid_y) not in self.background_matrix:
16                    extracted.append(p)
17
18            # Trigger Logic
19            if len(extracted) > TRIGGER_THRESHOLD:
20                self.is_stacking = True # Vehicle Entering
21                self.current_extracted.extend(extracted)
22            elif self.is_stacking and (timeout > IDLE_TIMEOUT):
23                # Vehicle Left -> Send to Queue
24                processing_queue.put((list(self.current_extracted), ts))
25
26            self.is_stacking = False

```

Walkthrough:

- **State Machine:** The loop toggles `self.is_stacking`.
- **Active State:** When foreground points > 50, it starts recording.

- **Stacking:** It accumulates slices, calculating Z based on `VEHICLE_SPEED_MPS` (5 km/h).
- **Completion:** When silence lasts $> 0.8s$, it assumes the vehicle has passed, bundles the data, and pushes it to the `processing_queue` for the other thread to handle.